

Geometric Algebra: Game Project

Due on 6 Jan 2026

De Pauw Ivo, Delombaerde Flor

Yoran Schoonooghe

Contents

1	Introduction	3
2	Applications	4
2.1	Camera Rotation	4
2.2	Player Collision	5
2.3	Projectile Reflection	6
3	Appendix	7
3.1	Local Coordinate System	7
3.1.1	Forward	7
3.1.2	Up	8
3.1.3	Right	8

1 Introduction

The Game Project consists of a third person 3D game in the FlyTracer engine. The character is a sphere that can move around and collides with the planes in the scene. The player can control the camera to zoom in and out on the character sphere, and to orbit around it. The camera also collides with the planes in the scene. An additional feature is that the player can throw a projectile that bounces off the various planes.

The goal is to hit a green target sphere with the projectile, after which it spawns to a new location. Each time the player hits the target, their score increases and the target radius decreases. If they hit the red target sphere however, they lose one point. The following sections cover the implementation of the described functionality, highlighting the use of Plane-based Projective Geometric Algebra (PPGA).

2 Applications

The applications cover the practical approach of this project, the steps that are necessary to create the functionality of the game. There are three applications in this section, namely the camera rotation, the player collision with the planes, and the reflection of the projectile when it hits a plane.

2.1 Camera Rotation

The camera can rotate around the local up direction and around the local right direction. These are called the camera yaw and pitch, respectively. Rotation is defined by Equation (1) where the rotator $R_{\phi I_o}$ is given by Equation (2). The rotator is calculated from the rotation angle ϕ and the normalized rotation axis I_o as the intersection between two origin planes.

$$rot(x)_{\phi I_o} = R_{\phi I_o} x R_{\phi I_o}^{-1} \quad (1)$$

$$R_{\phi I_o} = \cos\left(\frac{\phi}{2}\right) - \sin\left(\frac{\phi}{2}\right) I_o \quad (2)$$

The exponential expression for the rotator with normalized rotation axis $\check{I}$ is defined by Equation (3).

$$R_{\phi \check{I}} = e^{-\frac{1}{2} \phi \check{I}} \quad (3)$$

Applying Equation (1) and Equation (3) with the local up direction results in the camera yaw for an arbitrary rotation angle ϕ , given by Equation (4) and Equation (5).

$$yaw = rot(X_{camera\ eye})_{\phi\ up} = R_{\phi\ up} X_{camera\ eye} R_{\phi\ up}^{-1} \quad (4)$$

$$R_{\phi\ up} = e^{-\frac{1}{2} \phi\ up} \quad (5)$$

Following the same methodology, the camera pitch for a rotation angle ϕ is defined by Equation (6) where the rotator is given by Equation (7).

$$pitch = rot(X_{camera\ eye})_{\phi\ right} = R_{\phi\ right} X_{camera\ eye} R_{\phi\ right}^{-1} \quad (6)$$

$$R_{\phi\ right} = e^{-\frac{1}{2} \phi\ right} \quad (7)$$

2.2 Player Collision

To create an interactive environment for the player, the player should collide with the other elements surrounding them. The first collision a player encounters is typically the collision between the player and the ground plane. Since the player is represented by a sphere, the collision between the player and the surrounding planes can be classified as point-plane collisions. Equation (8) defines the interdistance between a point and a plane. The interdistance is the signed distance between the normalized trivector $\check{X}$ and the normalized vector $\check{n}$.

$$d(\check{X}, \check{n}) = \check{X} \vee \check{n} \quad (8)$$

Applying Equation (8) to calculate the distance between the player position X_{player} and a plane n results in Equation (9). If the interdistance $d(\check{X}_{player}, \check{n})$ becomes smaller than the player radius r_{player} , then the player sphere is colliding with the plane.

$$d(\check{X}_{player}, \check{n}) = \check{X}_{player} \vee \check{n} \quad (9)$$

To prevent the player sphere from clipping through the planes, a collision should invoke a translation of the player. A translation is a sandwich product, defined by Equation (10), where $T_{\bar{t}}$ is the translator. The translator is defined by Equation (11), with the translation vector offset $\bar{t}$.

$$trans(x)_{\bar{t}} = T_{\bar{t}} x T_{\bar{t}}^{-1} \quad (10)$$

$$T_{\bar{t}} = 1 - \frac{1}{2} \bar{t} (-e_0) \quad (11)$$

A translation can also be interpreted as a rotation around a vanishing axis U_{∞} . Combining this with the translator's exponential expression leads to Equation (12), where $|\bar{t}|$ is the translation amount. The vanishing axis U_{∞} can be retrieved by taking the dual of the bivector direction in which the translation should occur.

$$T_{\bar{t}} = e^{-\frac{1}{2} |\bar{t}| U_{\infty}} \quad (12)$$

The translation amount for the translation invoked by the collision is the difference between the player radius r_{player} and the interdistance, derived from Equation (9). The translation direction is defined by the bivector from the plane towards the player position. This bivector, known as a contraction line C , can be constructed from the extended dot product of the plane vector n and the player position trivector, represented by Equation (13).

$$C = n \cdot X_{player} \quad (13)$$

Combining Equation (10) and Equation (12) with the translation amount and direction for the collision results in the translation defined by Equation (14), where the collision translator T_c is defined by Equation (15).

$$trans(X_{player})_c = T_c X_{player} T_c^{-1} \quad (14)$$

$$T_c = e^{-\frac{1}{2} (r_{player} - d(\check{X}_{player}, \check{n})) \mathcal{H}(C)} \quad (15)$$

2.3 Projectile Reflection

The player can throw a projectile, a sphere, that bounces around. The collision when the projectile hits a wall plane is the same implementation as the player sphere. However, the ball bounces off the plane instead. With PPGA, this can be implemented by reflecting the direction over the plane it hits. Equation (16) defines the reflection of a bivector L on a reflector vector m .

$$ref(L)_m = -m L m^{-1} \quad (16)$$

Applying Equation (16) to the direction of the projectile leads to Equation (17), where $L_{projectile\ in}$ is the direction of the incoming projectile and $L_{projectile\ out}$ is the outgoing direction after the projectile hit the plane m .

$$L_{projectile\ out} = ref(L_{projectile\ in})_m = -m L_{projectile\ in} m^{-1} \quad (17)$$

Figure 1 visualizes this line-plane reflection. The red bivector is the direction of the incoming projectile, and the green plane is the reflector vector. The blue bivector is the resulting direction from the reflection.

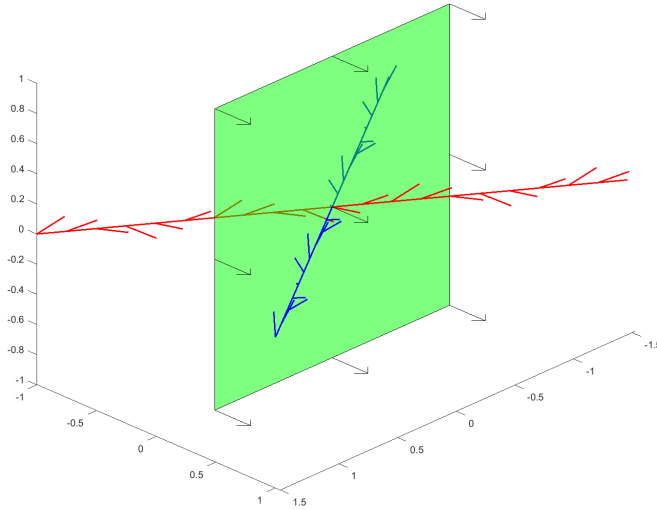


Figure 1: Line-Plane Reflection

3 Appendix

The appendix contains an additional application, namely the implementation of the local coordinate system. Due to its broad scope, this application was excluded from Section 2.

3.1 Local Coordinate System

One of the requirements to create player movement is a local coordinate system. This can be implemented in various ways, depending on the type of camera perspective. For a third person perspective, the local coordinate system depends on the camera eye and the camera target, the player character.

3.1.1 Forward

The local forward direction can be retrieved by normalizing the bivector from the camera position to the player character position, projected onto the ground plane. The first step is to create this bivector from the camera eye to the camera target. Equation (18) shows the join operation of trivector P and trivector Q . This results in a bivector L , oriented from trivector P to trivector Q .

$$L = P \vee Q \quad (18)$$

Applying Equation (18) to the camera eye and the camera target results in the bivector from the camera eye to the camera target, defined by Equation (19).

$$L_{eye\ to\ target} = X_{camera\ eye} \vee X_{camera\ target} \quad (19)$$

The second step is to project the bivector $L_{eye\ to\ target}$ on the ground plane. Equation (20) defines the projection of element X on element m , where X and m can be of any grade. This property is called the container property. Suppose X is a trivector and m is a vector, then the projection of X to m is the projection of a point on a plane.

$$proj(X)_m = (X \cdot m) m^{-1} \quad (20)$$

The projection from Equation (20) can be applied to project the bivector $L_{eye\ to\ target}$ on the ground plane m_{ground} . This is shown in Equation (21).

$$L_{||} = proj(L_{eye\ to\ target})_{m_{ground}} = (L_{eye\ to\ target} \cdot m_{ground}) m_{ground}^{-1} \quad (21)$$

The third step to create the forward direction is to normalize the projected bivector. A bivector can generally be defined by Equation (22), with $l_1, l_2, l_3, \mu_1, \mu_2$ and μ_3 in $\mathbb{R}$.

$$|L| = (l_1 \bar{e}_{23} + l_2 \bar{e}_{31} + l_3 \bar{e}_{12}) + (\mu_1 \bar{e}_{01} + \mu_2 \bar{e}_{02} + \mu_3 \bar{e}_{03}) \quad (22)$$

For any bivector L , a normalized bivector $\check{L}$ can be defined by Equation (23), where the norm $|L|$ is defined by Equation (24).

$$\check{L} = \frac{L}{|L|} \quad (23)$$

$$|L| = \sqrt{\langle L L^\dagger \rangle_0} = \sqrt{l_1^2 + l_2^2 + l_3^2} \quad (24)$$

Combining Equation (23) with the projected bivector $L_\parallel$ results in the normalized projected bivector $\check{L}_\parallel$, defined by Equation (25).

$$\check{L}_\parallel = \frac{L_\parallel}{|L_\parallel|} \quad (25)$$

Additionally, there is a final step to translate the forward direction to the pivot point of the player character. In many cases, the pivot point is at the bottom of the mesh and then this step may not be necessary. Since the player is represented by a sphere however, it makes more sense to move the pivot point to the sphere's center. On top of that, the pivot point may not be in the ground plane if the player jumps for instance. This step does not have a direct effect on translating the player character, but it does affect the behaviour of orbiting the camera around the player. Considering the container property of Equation (20), the forward direction can be translated by projecting the bivector on the camera target. This gives the resulting forward direction, defined by Equation (26).

$$forward = proj(\check{L}_\parallel)_{X_{camera\ target}} = (\check{L}_\parallel \cdot X_{camera\ target}) X_{camera\ target}^{-1} \quad (26)$$

3.1.2 Up

The up direction could be simplified to use the world up, being $\bar{e}_{31}$. However, this does not guarantee orthogonality between the local up and forward. A more robust approach is to create the local up direction from the ground plane and the player character position.

The first step is to construct the up direction as the contraction line from the ground plane to the player character position. This bivector $L_{ground\ to\ target}$ is the result of the extended dot product, given by (27).

$$L_{ground\ to\ target} = m_{ground} \cdot X_{camera\ target} \quad (27)$$

The second and final step is to normalize this bivector. Similarly to the forward direction, Equation (23) can be combined with $L_{ground\ to\ target}$ to get the normalized up direction, defined by Equation (28).

$$up = \frac{L_{ground\ to\ target}}{|L_{ground\ to\ target}|} \quad (28)$$

Alternatively, the up direction could be constructed from the rejection of $L_{eye\ to\ target}$. This follows the same methodology as the forward direction, though it introduces an extra complexity as the sign of the rejection depends on the relative position of the camera eye to the camera target.

3.1.3 Right

With the forward and up direction defined, the right direction is retrieved from the geometric product of the up and the forward directions. This geometric product, represented in Equation (29), returns a bivector orthogonal to both the forward and the up direction. Considering the right-handed system, the up direction needs to be the first element in the geometric product.

$$right = up * forward \quad (29)$$